

Student Name	Balaji MJ
Register Number	TU-6253210211004
Project Title	Unsupervised Financial Anomaly Detection via Isolation Forest from First Principles
Week	Week 7

WEEK-7 PROJECT

Task Name:

Unsupervised Financial Anomaly Detection via Isolation Forest from First Principles

Description:

This Week-7 task focuses on implementing an Isolation Forest anomaly detection algorithm from scratch. The objective is to detect unusual financial transactions without relying on labelled fraud data. Students must implement the recursive binary partitioning logic, construct an ensemble of isolation trees, measure the path length required to isolate each transaction, and convert the average path length into a mathematical anomaly score.

The project should demonstrate the complete pipeline from transaction data preparation to anomaly scoring, threshold selection, result interpretation, and performance analysis. The emphasis is on understanding the algorithm internally rather than simply calling a ready-made machine learning implementation.

1. PROJECT INPUT: FINANCIAL TRANSACTION DATA

A transaction record can contain numerical and categorical attributes describing financial behaviour.

- transaction_id
- timestamp
- customer_id
- transaction_amount
- account_balance
- transaction_frequency
- merchant_category
- location or distance-related feature
- device or channel indicator

Example transaction: Amount = ₹75,000, account balance = ₹1,20,000, transaction frequency = 18 per day, distance from usual location = 650 km.

2. UNDERSTANDING FINANCIAL ANOMALIES

A financial anomaly is a transaction or behavioural pattern that differs substantially from the normal population. An anomaly is not automatically fraud. It is a transaction that deserves further investigation because its feature combination is unusual relative to the data used to build the model.

Examples of potentially anomalous behaviour

- A transaction amount far above a customer's normal spending range.
- A sudden increase in transaction frequency within a short period.
- A transaction from a location that is highly unusual for the customer.
- A combination of high amount, unusual device, and unusual merchant category.
- A sudden account balance change that differs from historical behaviour.

Why use unsupervised learning?

Financial datasets may contain very few confirmed fraud cases compared with legitimate transactions. Isolation Forest can therefore be useful when reliable anomaly labels are unavailable or incomplete. The method learns the structure of the available feature space and identifies observations that are easier to isolate.

Core idea of Isolation Forest

Instead of modelling normal observations directly, Isolation Forest repeatedly partitions the data using random feature selections and random split values. An unusual observation tends to be separated from the rest of the data with fewer partitions. Therefore, anomalous observations usually have shorter average path lengths in the isolation trees.

Project objective

The main objective is to build an Isolation Forest from first principles and use the resulting mathematical score to identify high-risk or unusual financial transactions for further investigation.

3. DATA PREPROCESSING AND FEATURE ENGINEERING

Before constructing isolation trees, the transaction data should be cleaned and converted into a numerical feature matrix. The preprocessing stage should preserve the meaning of the financial variables while preventing invalid records from affecting the model.

Data cleaning steps

- Remove exact duplicate transaction records where appropriate.
- Handle missing numerical values using a documented strategy such as median imputation.
- Handle missing categorical values with a dedicated category such as Unknown.
- Convert timestamps into useful features such as hour, day of week, or time since previous transaction.
- Check for impossible or invalid values such as negative amounts when the business rule does not permit them.
- Keep an audit record of preprocessing decisions.

Recommended numerical features

Feature	Meaning	Example
transaction_amount	Value of the transaction	■5,250
account_balance	Balance associated with transaction	■84,000
daily_txn_count	Number of transactions in a day	12
avg_amount_7d	Recent average transaction amount	■2,450
amount_deviation	Difference from recent spending pattern	■18,700
distance_from_usual	Distance from normal transaction location	320 km
hour	Hour extracted from timestamp	02

Feature matrix

After preprocessing, represent the data as a matrix X with n transactions and d numerical features:

$$X = [x_1, x_2, \dots, x_n], \text{ where each } x_i = [x_{i1}, x_{i2}, \dots, x_{id}]$$

The first-principles implementation should operate on these feature vectors and should not depend on a built-in Isolation Forest class for the core tree construction and scoring logic.

4. ISOLATION TREE CONSTRUCTION

An Isolation Tree, or iTree, is a binary tree created by recursively partitioning a subset of transactions. At each internal node, a feature is selected randomly and a split value is selected randomly between the minimum and maximum values of that feature in the current subset.

Recursive partitioning logic

- Start with a sample of transactions at the root node.
- If the current subset contains one transaction, stop and create an external node.
- If the maximum tree depth is reached, stop and create an external node.
- Randomly choose one feature whose values are not constant in the current subset.
- Choose a random split value between the minimum and maximum value of the selected feature.
- Partition the records into left and right subsets.
- Recursively repeat the process for both subsets.

Pseudocode

```
build_tree(X, current_depth, max_depth):
    if len(X) <= 1 or current_depth >= max_depth:
        return ExternalNode(size=len(X))

    choose random feature q
    if max(X[q]) == min(X[q]):
        return ExternalNode(size=len(X))

    split = random_uniform(min(X[q]), max(X[q]))

    left = X[X[q] < split]
    right = X[X[q] >= split]

    return InternalNode(
        feature=q,
        split=split,
        left=build_tree(left, current_depth+1, max_depth),
        right=build_tree(right, current_depth+1, max_depth)
    )
```

Important implementation rule

If all values of a selected feature are identical inside the current node, that feature cannot produce a useful split. The implementation should select another variable or terminate the node when no valid split exists.

5. BUILDING THE ISOLATION FOREST

An Isolation Forest is an ensemble of many independently constructed isolation trees. Using multiple trees reduces the dependence on any single random partitioning sequence.

Forest construction

- Choose the number of trees, for example 100 or 200.
- For each tree, randomly sample a subset of the training transactions.
- Build an isolation tree from the sampled subset.
- Store the tree structure, including selected feature, split value, child nodes, and external node size.
- Repeat until the required number of trees is created.

Maximum sample size and tree height

For a subsample size ψ , a practical tree height limit is approximately $\text{ceil}(\log_2(\psi))$. The exact implementation should document the selected subsample size and stopping rule.

Example forest configuration:

Parameter	Illustrative value
Number of trees	100
Subsample size ψ	256
Maximum depth	$\text{ceil}(\log_2(256)) = 8$
Split selection	Random feature + random split value
Output	Average path length + anomaly score

Why an ensemble is useful

A single random tree can produce a path length that is strongly affected by its random choices. A forest averages the isolation behaviour over many trees, producing a more stable estimate for each transaction.

6. PATH LENGTH AND MATHEMATICAL ANOMALY SCORE

For a transaction x , the path length $h(x)$ is the number of edges traversed from the root of an isolation tree until x reaches an external node. Transactions that are isolated quickly have shorter path lengths.

Average path length

For a forest containing t trees, calculate the path length in every tree and take the mean:

$$E(h(x)) = (1/t) \times \sum_{i=1}^t h_i(x), \text{ for } i = 1 \text{ to } t$$

External-node adjustment

When a tree stops because a node contains more than one observation, the remaining expected path length can be adjusted using the average path length of an unsuccessful search in a binary search tree:

$$c(n) = 2H(n-1) - 2(n-1)/n \text{ for } n \geq 2$$
$$c(1) = 0 \text{ and } c(2) = 1$$

Here $H(n-1)$ is the harmonic number. In an implementation, $H(k)$ can be calculated directly for smaller k or approximated using a logarithmic expression for larger values.

Isolation Forest anomaly score

The standard Isolation Forest scoring relationship can be represented as:

$$s(x, \psi) = 2^{(-E(h(x)) / c(\psi))}$$

The score moves toward 1 when a transaction is isolated in fewer steps and moves toward 0 when its path length is closer to the expected path length of a random observation. The exact threshold used to flag observations should be documented and validated against the project's operational requirements.

Interpretation

Observed behaviour	Path length	Typical score interpretation
Easy to isolate	Short	More anomalous
Near expected isolation	Medium	Less distinctive
Hard to isolate	Long	More typical

7. SCORING EVERY FINANCIAL TRANSACTION

After the forest is constructed, every transaction is passed through every isolation tree. The implementation records the path length for each tree, calculates the average path length, and then computes the anomaly score.

Scoring workflow

- Take one transaction x .
- Traverse Tree 1 until an external node is reached.
- Record the adjusted path length.
- Repeat for all trees.
- Calculate the mean path length $E(h(x))$.
- Apply the anomaly-score formula.
- Repeat for every transaction.
- Store transaction_id, average path length, anomaly score, and anomaly flag.

Illustrative result table

Transaction	Avg. Path	Anomaly Score	Flag
TXN-1001	7.4	0.42	Normal
TXN-1002	3.1	0.79	Anomaly
TXN-1003	6.8	0.48	Normal
TXN-1004	2.7	0.84	Anomaly
TXN-1005	7.1	0.45	Normal

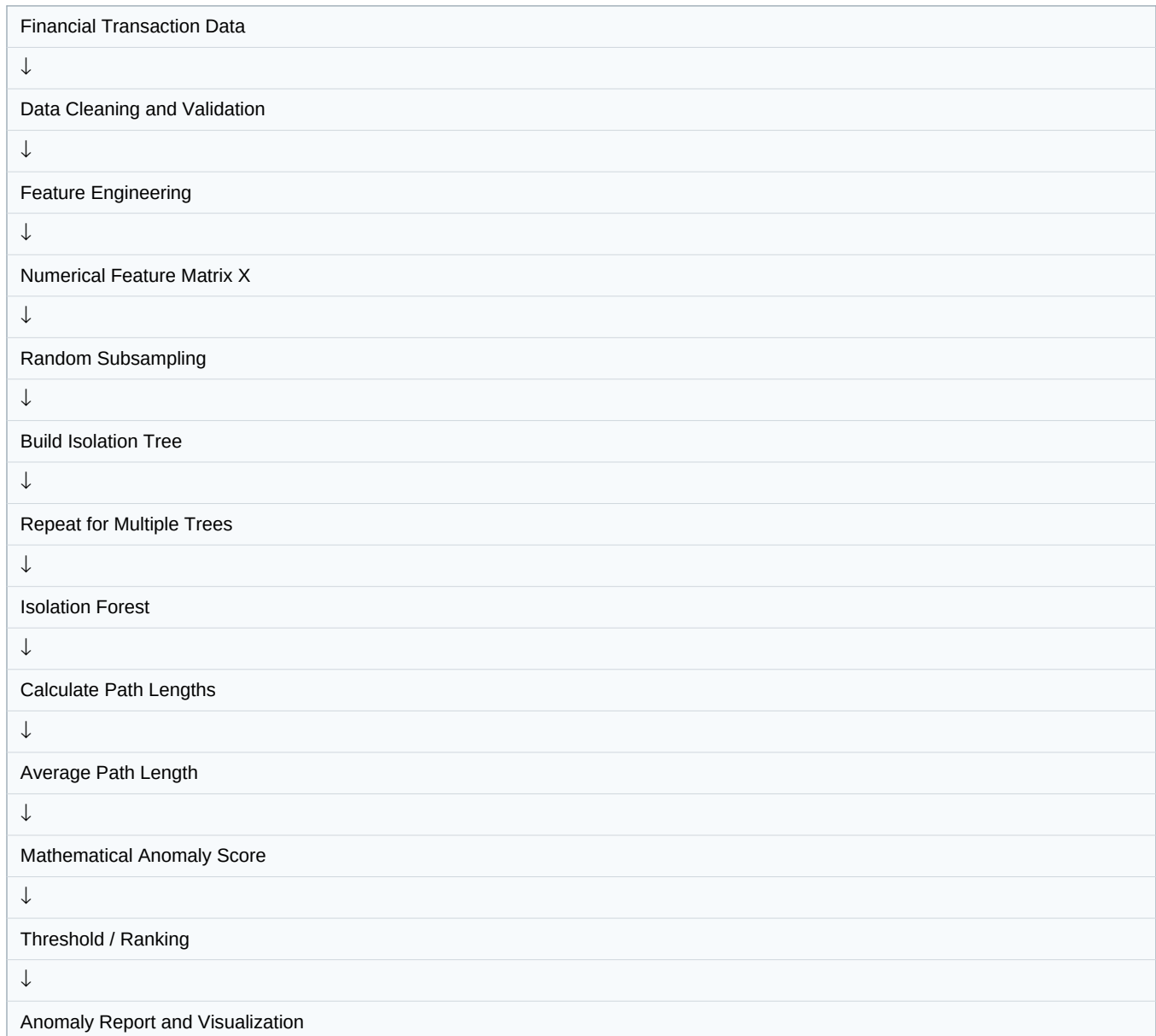
The numerical values above are illustrative only. The completed project must calculate the actual values from the implemented forest and selected dataset.

Anomaly threshold

A threshold can be selected using a project-defined contamination assumption, score distribution, percentile rule, or domain review process. The report should explain why the selected threshold was used rather than treating every high score as confirmed fraud.

8. END-TO-END SYSTEM ARCHITECTURE

The complete workflow should connect data preparation, first-principles tree construction, ensemble scoring, and reporting.



Recommended technology

- Python as the primary programming language.
- NumPy for numerical arrays and mathematical operations.
- Pandas or Polars for transaction preprocessing and result handling.
- Matplotlib or Plotly for score distributions and transaction visualizations.
- CSV or Parquet for input and output storage.

First-principles requirement

The central tree-building and scoring components should be implemented manually. Libraries may be used for data loading, array operations, plotting, and validation, but the project should demonstrate the student's own recursive partitioning, tree traversal, path-length, and anomaly-score logic.

9. PERFORMANCE, VALIDATION AND VISUALIZATION

Performance analysis

- Measure preprocessing time.
- Measure time required to build the isolation forest.
- Measure scoring time for the complete transaction set.
- Record peak or approximate memory usage.
- Compare runtime when the number of trees or sample size changes.

Validation checks

- Verify that every tree has valid left and right branches where a split is created.
- Verify that path lengths are non-negative.
- Verify that $c(n)$ is handled correctly for external nodes.
- Verify that anomaly scores are within the expected mathematical range.
- Check that the same random seed can reproduce the same experiment when reproducibility is required.
- Inspect a sample of highly anomalous records manually.

Recommended visualizations

- Histogram of anomaly scores.
- Scatter plot of transaction amount versus anomaly score.
- Top-N transactions by anomaly score.
- Score distribution by merchant category.
- Score distribution by transaction hour.
- Average score by customer or account, when appropriate.

Example reporting table

Metric	Value to report
Number of transactions	Actual dataset size
Number of features	d
Number of trees	t
Subsample size	ψ
Maximum depth	Configured limit
Build time	Measured seconds/minutes
Scoring time	Measured seconds/minutes
Anomaly threshold	Selected value/rule
Flagged transactions	Count and percentage

10. FINAL PROJECT DELIVERABLES

The completed Week-7 project should contain the following deliverables:

1. Data loading and preprocessing pipeline.
2. Feature engineering for financial transaction attributes.
3. A first-principles implementation of recursive isolation-tree construction.
4. An ensemble implementation for building multiple isolation trees.
5. Tree traversal logic for calculating transaction path lengths.
6. External-node path-length adjustment using $c(n)$.
7. Average path length calculation across the forest.
8. Mathematical Isolation Forest anomaly score calculation.
9. Anomaly threshold or ranking procedure.
10. Final transaction-level anomaly report.
11. Visualizations of anomaly scores and important transaction patterns.
12. Runtime and memory analysis.
13. Reproducibility information such as random seed and experiment configuration.

PROJECT OBJECTIVE

The main objective of this project is to understand and implement unsupervised anomaly detection for financial transactions using Isolation Forest from first principles. By repeatedly and randomly partitioning the feature space, the method identifies observations that can be isolated with relatively short paths. The resulting mathematical anomaly score can then be used to rank unusual transactions for further analysis.

FINAL OUTPUT

Output	Description
Transaction	Unique transaction identifier
Average Path Length	Mean isolation depth across trees
Anomaly Score	Mathematical Isolation Forest score
Anomaly Flag	Normal or anomaly according to selected rule
Rank	Relative anomaly position

Note: Any numerical examples in this document are illustrative. Final scores, thresholds, runtime values, and anomaly counts must be generated from the actual dataset and the student's implementation.